

查找 J, K, 7 的几种方式

- Q1: 通过符号表示/找 FF 设计 Sequential Circuit
- 1. 看图, 找对应关系
 - 2. 列出每个 Z, Y, Dn+1, ① 等的公式 $\Rightarrow$ 用 FF 公式 (some)
 - 3. 列出上述参数满足 1 要求的 所需值 $\Rightarrow$ 列出表格 (state table)
 - 4. 用 K-map 画对应值时, 直接 整列 取 (白: cur state, input 因: FF) (但注意 X, A, B, X 顺序, 以及 1, 0 的顺序)

第 2. 注意 NAND, A, B, A', B', 对应关系

Q2: 设计 Circular Circuit

- 1. 利用 truth table
- 2. 找 Dn+1 与 Dn, J, K 等的关系

Q3: SRAM > DRAM > SSD > HDD > PROM

- Q4: ~~XXXX~~ (Hword 由多 bit 组成)
- data line: 每个 word 单元的 bit 数量
 - address line: 需要 2 个 word $\Rightarrow$ 7

起点画双圈

Lec 1 CPU, Memory, I/O device

123, 23 (decima) $\rightarrow$ binary

2 1 2 3
2 1 6 1
2 1 3 0
2 1 1 5
2 1 7 1
2 1 3 1
2 1 2 1
2 1 4 0
2 :

4. ~~XXXX~~ { 逻辑运算: 零拓展 ex: ori, 计算: 符号拓展 (根据最高位拓展) ex: addi, lw, sw }

5. 1) ori \$b, \$0, 0x2 $\Rightarrow$ load function

2) andi \$b, \$0, 0x2

只有同时为 1 时, 才为 1

3) xori \$b, \$0, 0x2 异或

只有不同时, 才为 1

4) nori \$b, \$0, 0x2 $\Leftrightarrow$ 刚好

只有同时为 0 时, 才为 1

6. 溢出检测
- unsigned: 溢出就是溢出
 - signed: { 同+同=异 溢出
同+异 未溢出

7. lui d, const: 将 const (16 bits) 放入 d 的前 2 个 bytes (高 16 位)

ori d, d, imm: 将 immediate 与 d 再进行 or 运算

8. branch delay slot $\Rightarrow$ PC 自动下一句 $\Rightarrow$ 使用 No-op

Lec 3

1. push

大地址

小地址

subu \$sp, \$sp, 4

sw \$t0, 0(\$sp)

lw \$t0, 0(\$sp)

addu \$sp, \$sp, 4

2. pop

大地址

小地址

The jal and jr Instruction

◆ jal sub

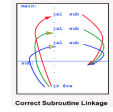
- (1) set \$ra to PC + 4 (see next slide for the details)
- the result is that \$ra stores the address of the (n+2)th instruction (suppose jal is the nth instruction)

- in memory: **jal** branch delay slot | **instruction** that will return to

- (2) set PC to sub

◆ jr \$ra

- set PC to \$ra



Calling Convention — Use of Registers

◆ Different roles of registers

- \$t0 - t9: subroutines can use without caution
- \$s0 - \$s7: saved registers; subroutines should not use (if need to use it, **save the value first** and restore the value after use)
- \$a0 - \$a3: contain arguments for the subroutine (用于传参数)
- \$v0 - \$v1: contain values returned from the subroutine (用于返回结果)

1. switching function: truth table $\Rightarrow$ switching function

◆ Consider switching functions defined over N variables, how many possible switching functions could we have?

- 2. 1) Absorption Theorem $A(A+B) = A$; $A+AB = A$
- 2) Consensus Theorem $AB + \bar{A}C + BC = AB + \bar{A}C$

Name	Graphical Symbol	Algebraic Function	Truth Table															
AND		$F = A \cdot B$ or $F = AB$	<table><tr><td>A</td><td>B</td><td>F</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	F	0	0	0	0	1	0	1	0	0	1	1	1
A	B	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$F = A + B$	<table><tr><td>A</td><td>B</td><td>F</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	F	0	0	0	0	1	1	1	0	1	1	1	1
A	B	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
NOT		$F = \bar{A}$ or $F = A'$	<table><tr><td>A</td><td>F</td></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	F	0	1	1	0									
A	F																	
0	1																	
1	0																	
NAND		$F = \overline{A \cdot B}$	<table><tr><td>A</td><td>B</td><td>F</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	1	0	1	1	1	0	1	1	1	0
A	B	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$F = \overline{A + B}$	<table><tr><td>A</td><td>B</td><td>F</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	1	0	1	0	1	0	0	1	1	0
A	B	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
XOR		$F = A \oplus B$	<table><tr><td>A</td><td>B</td><td>F</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	0	0	1	1	1	0	1	1	1	0
A	B	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																

Simplify: $(A + C)(AD + \bar{A}D) + AC + C$:

Expression
 $(A + C)(AD + \bar{A}D) + AC + C$
 $(A + C)A(D + \bar{D}) + AC + C$
 $(A + C)A + AC + C$
 $A(A + C) + C + C$
 $A + AC + C$
 $A + (A + C)C$
 $A + C$

Rule(s) Used
Original Expression
Distributive, 分配律
Complement, Identity.
Commutative, Distributive.
Associative, Idempotent.
Distributive, 需要律
Idempotent, Identity, Distributive.
Identity, twice.

Using K-map to simplify POS form

◆ Difference

- Group the '0' miniterms
- Note the mapping: 0101 $\rightarrow$ **A+B+C+D'**
- Simplify: $POS = \prod(m_i, m_j, \dots)$

Out = $(A+B+C+D')(A+B+C+D)(A+B+C+D)(A+B+C+D)$
 $(\bar{A}+\bar{B}+\bar{C}+D)(\bar{A}+B+C+D)(\bar{A}+B+C+D)$

$f = ABC + ABC + ABC$
 $\Rightarrow f = (A+B+C)(A+B+C)(A+B+C)$
 $= MAMM (M \text{ rotation}) = 11, 10, 1, 2$

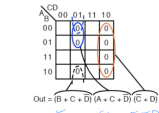
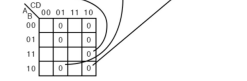
Row No.	ABC	Minterms m_i	Minterms m_i
0	000	$m_0 = A'B'C'$	$m_4 = A'B+C'$
1	001	$m_1 = A'B'C$	$m_5 = A'B+C$
2	010	$m_2 = A'BC'$	$m_6 = A+B'C'$
3	011	$m_3 = A'BC$	$m_7 = A+B'C$
4	100	$m_4 = A'B'C'$	$m_0 = A'B+C'$
5	101	$m_5 = A'B'C$	$m_1 = A'B+C$
6	110	$m_6 = A'BC'$	$m_2 = A+B'C'$
7	111	$m_7 = A'BC$	$m_3 = A+B'C$

Using K-map to simplify POS form

Out = $(A+B+C+D)(A+B+C+D)(A+B+C+D)(A+B+C+D)$
 $(\bar{A}+\bar{B}+\bar{C}+D)(\bar{A}+B+C+D)(\bar{A}+B+C+D)$

$f = ABCD + \bar{A}BCD + A\bar{B}CD + AB\bar{C}D$

Out = $(A+B+C+D)(A+B+C+D)(A+B+C+D)(A+B+C+D)$
 $(\bar{A}+\bar{B}+\bar{C}+D)(\bar{A}+B+C+D)(\bar{A}+B+C+D)$



The red group: A and B got cancelled. CD = 10 $\rightarrow$ C'D

Lec 6

1. Quine - McCluskey Table { next state = f(cur state, input)
output = g(cur state, input)

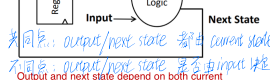
2. Asynchronous Counter — Ripple Counter

Lec 7

1. FSM = {S, I, O, π } S, I, O 都是 set

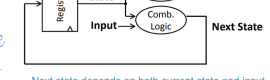
current state + input function $\rightarrow$ output + next state
 $S + I \rightarrow \pi \rightarrow D + \text{next state}$

2. 2 models



不同点: output/next state 都 depend on current state and input

1) Mealy Machine

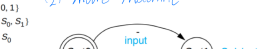


Next state depends on both current state and input. However, output depends only on current state



自环: 0/0: S0 $\rightarrow$ S1 I=0 Snext=S1 O=0

0/1: S0 $\rightarrow$ S1 I=1 Snext=S1 O=0



1) Moore Machine

FLIP-FLOP NAME	FLIP-FLOP SYMBOL	CHARACTERISTIC TABLE			EQUATION
SR		S	R	Q(next)	$Q_{next} = S + R'Q$ SR = 0
		0	0	Q	
		0	1	0	
		1	0	1	
		1	1	?	
JK		J	K	Q(next)	$Q_{next} = JQ' + K'Q$
		0	0	Q	
		0	1	0	
		1	0	1	
		1	1	Q'	
D		D	Q(next)	$Q_{next} = D$	
		0	0		
		1	1		
T		T	Q(next)	$Q_{next} = TQ'$	
		0	Q		
		1	Q'		

Memory Hierarchy and Cache

- 1. Characteristics of different memory units
- 1. CPU execution cycle = fetch cycle + execution cycle
- 2. Access Method $\Rightarrow$ 通过什么大的配合, 但要求又快又大的 memory
- 4) Sequential Access (ex: magnetic tape) } data 按位, 存取慢
- (2) Direct Access (ex: secondary device (disk)) } data 按位, 存取慢
- (3) Random Access (ex: RAM) 直接访问对应 address 取 data

3. Access Time

- 2) Memory Cycle Time $\Rightarrow$ 仅针对 RAM
- access time + additional time (2 次: 访问间隔)
- 3) Transfer Rate = $\frac{\text{memory cycle time}}{\text{unit}}$ 单位: MT/s (million transfer/s)

- 4. { volatile: 需要一直充电, 断电丢失 data register, RAM
- { non-volatile: 不用一直充电 hard drive tertiary
- { erasable: 可更改 register, hard drive
- { non-erasable: 不可更改 ROM (read only memory) EX: 打开 computer
- { internal memory: 一些重要的仍操作 MM/register/cache
- { external memory $\Rightarrow$ 需通过 I/O controller 间接访问 disk

5. Hierarchy { Expose Hierarchy: programmer 决定 data best-allocation

Hidden Hierarchy: memory system behind 决定 data movement

6. Temporal Locality: the same thing again soon

Spatial Locality: something near that thing soon

7. Locality of Reference 局部性原理 $\Rightarrow$ clustering effect 聚集效应

1) cluster 如何形成? 若在同一时间上, 访问了地址 X;

那么在此之后此时间内, 会连续访问 (X, X+1) 之间地址

2) cluster 如何形成? 若在同一时间上, 访问了地址 X;

那么在此之后此时间内, 会连续访问 (X, X+1) 之间地址

Tag Size	Smaller	Larger
SRAM Overhead	Less	More
Controller Logic	Less	More
Speed	Faster	Slower
Price	Less	More
Scalability	Very	Not Very
# of conflict misses	Lots	Zero
Hit Rate	Low	High

8. Eviction Policies 缓存替换策略

1) Direct Mapping 因为 fixed allocation $\Rightarrow$ 直接替换

2) Associative Mapping: ① FIFO ② LRU ③ Random

2) write: ① cache, disk 同步更新 ② dirty bit ③ set 3

S + RQ	0	0	0	X
	0	1	1	0
	1	0	0	1
	1	1	X	0
Q + KQ	Q	Qnext	J	K
	0	0	0	X
	0	1	1	X
	1	0	X	1
	1	1	X	0
= D	Q	Qnext	D	
	0	0	0	
	0	1	1	
	1	0	0	
	1	1	1	
Q + TQ	Q	Qnext	T	
	0	0	0	
	0	1	1	
	1	0	1	
	1	1	0	

